

项目转移

项目转移步骤

1. 删除多余文件

```
for dir in */; do find "$dir" -maxdepth 1 -mindepth 1 \( -type d ! -name "src" -  
exec rm -rf {} + -o -type f ! -name "pom.xml" -exec rm -f {} + \); done
```

带确认的安全版

`-WhatIf` 只显示会删什么，不真正删除。确认无误后去掉 `-WhatIf` 再执行

```
Get-ChildItem -Directory | ForEach-Object { Get-ChildItem $_.FullName | Where-  
Object { $_.Name -ne 'src' -and $_.Name -ne 'pom.xml' } | Remove-Item -Recurse -  
Force -WhatIf }
```

2. 压缩

把文件夹压缩，采用7z格式，体积越小越好
分片编码

```
# encode.ps1 - 文件分片Base64编码（输出txt）  
param(  
    [string]$FilePath,  
    [int]$ChunkSize = 800000,  
    [string]$OutputDir = "encoded_chunks"  
)  
  
if (-not (Test-Path $FilePath)) {  
    Write-Host "[错误] 找不到文件: $FilePath" -ForegroundColor Red; exit 1  
}  
  
New-Item -ItemType Directory -Force -Path $OutputDir | Out-Null  
$bytes = [IO.File]::ReadAllBytes($FilePath)  
$totalChunks = [Math]::Ceiling($bytes.Length / $ChunkSize)  
  
Write-Host "文件大小: $($bytes.Length) bytes | 原始分片: $ChunkSize bytes | 总分片:  
$totalChunks"  
  
for ($i = 0; $i -lt $totalChunks; $i++) {  
    $start = $i * $ChunkSize  
    $len = [Math]::Min($ChunkSize, $bytes.Length - $start)  
    $chunk = New-Object byte[] $len
```

```

[Array]::Copy($bytes, $start, $chunk, 0, $len)

$b64 = [Convert]::ToBase64String($chunk)
# ⬇ 关键修改：后缀改为 .txt
$outFile = Join-Path $OutputDir ("chunk_{0:D4}.txt" -f $i)
[IO.File]::WriteAllText($outFile, $b64)
Write-Host "  [$(($i+1)/$totalChunks)] -> $outFile (输出约 $($b64.Length) 字符)"
}

"$totalChunks`r`n$([IO.Path]::GetFileName($FilePath))" | Out-File -Encoding utf8
"$OutputDir\_meta.txt"
Write-Host "`n✅ 编码完成！输出目录: $OutputDir" -ForegroundColor Green

```

3. 解码(注意文件名和内容保持一致)

```

# decode.ps1 - Base64分片解码合并（读取txt）
param([string]$InputDir = ".")

$metaFile = Join-Path $InputDir "_meta.txt"
if (-not (Test-Path $metaFile)) {
    Write-Host "[错误] 未找到 _meta.txt, 请确认所有文件已复制完整" -ForegroundColor Red; exit 1
}

$lines = Get-Content $metaFile
$totalChunks = [int]$lines[0]
$originalName = $lines[1].Trim()
Write-Host "原始文件: $originalName | 预期分片: $totalChunks"

$ms = New-Object IO.MemoryStream
$missing = 0

for ($i = 0; $i -lt $totalChunks; $i++) {
    # 与 encode.ps1 输出格式严格一致
    $file = Join-Path $InputDir ("chunk_{0:D4}.txt" -f $i)
    if (-not (Test-Path $file)) {
        Write-Host "  [缺失] $file" -ForegroundColor Yellow; $missing++; continue
    }
    # 清洗可能混入的换行符、BOM头及空白字符
    $b64 = [IO.File]::ReadAllText($file).Trim() -replace '[\r\n\s]', ''
    try {
        $bytes = [Convert]::FromBase64String($b64)
        $ms.Write($bytes, 0, $bytes.Length)
        Write-Host "  [$(($i+1)/$totalChunks)] 已解码 ($( $bytes.Length) bytes)"
    }
}

```

```
    } catch {
        Write-Host " [损坏] $file Base64解码失败: $_" -ForegroundColor Red;
    $missing++
    }
}

if ($missing -gt 0) {
    Write-Host "`n❌ 存在 $missing 个缺失或损坏的分片，无法还原!" -ForegroundColor Red; exit 1
}

$outputPath = Join-Path $InputDir $originalName
[IO.File]::WriteAllBytes($outputPath, $ms.ToArray())
$finalSize = $ms.Length
$ms.Close()
Write-Host "`n✅ 还原成功: $outputPath ($finalSize bytes)" -ForegroundColor Green
```

批量初始化

```
for dir in */; do [ -d "$dir" ] && git init "$dir"; done
```

批量提交

```
find . -maxdepth 2 -name ".git" -type d | while read dir; do (cd "$(dirname "$dir")" && git add . && git commit -m "feat: 批量提交"); done
```